

Task-3: SQL For Data Analysis

Objective: Use SQL queries to extract and analyze data from a database.

Tools: MySQL or PostgreSQL or SQLite

Deliverables: SQL queries in a SQL file + screenshots of output

The screenshot shows a SQL client interface with a dark theme. The main window displays a SQL query and its results. The query is:

```
1 -- 1. View all data
2 SELECT * FROM cleaned_defence_data;
```

The results are displayed in a table with 8 columns: ID, Description, Code, A..., Revis..., Budget..., B..., Budget... The table contains 8 rows of data, each representing a different category of defence expenditure.

ID	Description	Code	A...	Revis...	Budget...	B...	Budget...
1.0	Pay and Allowances of the Army	2076	49647...	49647...	54446...		
2.0	Pay and Allowances and Miscellaneous Expenses of the Auxiliary For...	2076	967.53	967.53	1056...		
3.0	Pay and Allowances of Civilians	2076	4041.8	4041.8	4410...		
4.0	Transportation	2076	2676...	2676...	2368.0		
5.0	Military Farms	2076	494.96	494.96	428.77		
6.0	Ex-Servicemen Contributory Health Scheme	2076	1781...	1781...	1428...		
7.0	Inspection	2076	760.96	760.96	832.8		
8.0	Stores	2076	13955...	13955...	15198...		

The interface also includes a sidebar on the left with a list of databases (SQLite, MariaDB, PostgreSQL, MS SQL) and a History panel on the right showing the executed query and its timestamp (17:16:55).

2.

The screenshot shows a database management interface with a sidebar on the left containing a tree view with 'SQLite' selected. The main area displays a SQL query and its results. The query is:

```
1 -- 2. View limited rows (top 5 rows)
2 SELECT * FROM cleaned_defence_data
3 LIMIT 5;
```

The results are shown in a table with columns: ID, Description, Code, A, Revise, Budget, B, Budget. The data is as follows:

ID	Description	Code	A	Revise	Budget	B	Budget
1.0	Pay and Allowances of the Army	2076		49647..	49647..	56446..	
2.0	Pay and Allowances and Miscellaneous Expenses of the Auxiliary For...	2076		967.53	967.53	1056...	
3.0	Pay and Allowances of Civilians	2076		4041.8	4041.8	4410...	
4.0	Transportation	2076		2676...	2676...	2368.0	

The right sidebar shows a 'History' panel with two entries, each showing a SQL query and a timestamp.

3.

The screenshot shows the same database management interface as before, but with a different SQL query and results. The query is:

```
1 -- 3. View specific columns
2 SELECT
3   c3 AS Scheme,
4   CAST(c6 AS REAL) AS Budget_2015_2016,
5   CAST(c14 AS REAL) AS Budget_2016_2017
6 FROM cleaned_defence_data;
```

The results are shown in a table with columns: Scheme, Budget_2015_2016, Budget_2016_2017. The data is as follows:

Scheme	Budget_2015_2016	Budget_2016_2017
0	0	0
2076	49647.95	60548.10
2076	967.53	1172.2
2076	4041.8	4878.8
2076	2676.16	2828.1
2076	494.96	354.1
2076	1781.39	2639
2076	760.96	851.04
2076	13955.3	16697.84
2076	6327.95	8259
2076	4436.32	5453
2076	874.47	1010.39

The right sidebar shows a 'History' panel with three entries, each showing a SQL query and a timestamp.

4.

The screenshot shows a database management tool with a dark theme. On the left, there's a sidebar with a tree view showing a SQLite database named 'demo' with a table 'cleaned_defence_data'. The main area displays a SQL query:

```
1 -- 4. List unique values (Distinct Descriptions)
2 SELECT DISTINCT c2 AS Unique_Descriptions
3 FROM cleaned_defence_data;
```

The results are shown in a table with one column, 'Unique_Descriptions', containing the following values:

Unique_Descriptions
Description
Pay and Allowances of the Army
Pay and Allowances and Miscellaneous Expenses of the Auxiliary Forces
Pay and Allowances of Civilians
Transportation
Military Farms
Ex Servicemen Contributory Health Scheme
Inspection
Stores

On the right, a 'History' panel shows a list of executed queries with their timestamps.

5.

The screenshot shows the same database management tool. The SQL query in the main area is:

```
1 -- 5. Use WHERE to filter rows (where revised budget is greater than original)
2 SELECT
3   c3 AS Scheme,
4   CAST(c6 AS REAL) AS Budget_2015_2016,
5   CAST(c7 AS REAL) AS Revised_2015_2016
6 FROM cleaned_defence_data
7 WHERE CAST(c7 AS REAL) > CAST(c6 AS REAL);
```

The results are shown in a table with three columns: 'Scheme', 'Budget_2015_2016', and 'Revised_2015_2016'.

Scheme	Budget_2015_2016	Revised_2015_2016
0876	-1964.72	0

The 'History' panel on the right shows the sequence of queries executed, including the previous query from step 4 and the current query from step 5.

6.

SQLite

```

1 -- 6. Combine conditions with AND/OR
2 SELECT
3   c3 AS Scheme,
4   CAST(c6 AS REAL) AS Budget_2015_2016,
5   CAST(c14 AS REAL) AS Budget_2016_2017
6 FROM cleaned_defence_data
7 WHERE CAST(c6 AS REAL) > 5000 AND CAST(c14 AS REAL) > 10000;

```

Scheme	Budget_2015_2016	Budget_2016_2017
2076	49647.95	60548.18
2076	13955.3	16697.84
2076	85785.82	104158.95

History

SQLite

```

-- 6. Combine conditions with AND/OR
SELECT
  c3 AS Scheme,
  CAST(c6 AS REAL) AS Budget_2015_2
...
17:23:52

```

SQLite

```

-- 5. Use WHERE to filter rows (where rs
SELECT
  c3 AS Sch
...
17:23:17

```

SQLite

```

-- 4. List unique values (Distinct Descr
SELECT DISTINCT c2 AS Unique_Description
FROM clea
...
17:22:45

```

7.

SQLite

```

1 -- 7. Sort results in descending order (by budget increase)
2 SELECT
3   c3 AS Scheme,
4   CAST(c6 AS REAL) AS Budget_2015_2016,
5   CAST(c14 AS REAL) AS Budget_2016_2017,
6   (CAST(c14 AS REAL) - CAST(c6 AS REAL)) AS Increase
7 FROM cleaned_defence_data
8 ORDER BY Increase DESC
9 LIMIT 5;

```

Scheme	Budget_2015_2016	Budget_2016_2017	Increase
2076	85785.82	104158.95	18373.129999999999
2076	49647.95	60548.18	10900.230000000003
2076	13955.3	16697.84	2742.5400000000001
2076	6327.95	8259	1931.0500000000002
2076	4436.32	5453	1016.6800000000003

History

SQLite

```

-- 7. Sort results in descending order
SELECT
  c3 AS Scheme,
  CAST(c6 AS
...
17:25:40

```

SQLite

```

-- 6. Combine conditions with AND/OR
SELECT
  c3 AS Scheme,
  CAST(c6 AS REAL) AS Budget_2015_2
...
17:23:52

```

SQLite

```

-- 5. Use WHERE to filter rows (where rs
SELECT
  c3 AS Sch
...
17:23:17

```

8.

The screenshot shows a SQL editor interface with a dark theme. On the left, there's a sidebar with a tree view showing a database named 'SQLite' with a table 'cleaned_defence_data'. The main editor area contains a SQL query:

```
1 -- 8. Count total number of rows
2 SELECT COUNT(*) AS Total_Rows
3 FROM cleaned_defence_data;
```

The query is executed, and the result is displayed in a table with one row and one column:

Total_Rows
15

On the right, there's a 'History' panel showing the execution history of the query, including the SQL code and the execution time (17:26:08).

9.

The screenshot shows the same SQL editor interface. The main editor area contains a SQL query:

```
1 -- 9. Aggregate: Sum, Avg, Max, Min of 2016 Budget
2 SELECT
3   SUM(CAST(c14 AS REAL)) AS Total_2016,
4   AVG(CAST(c14 AS REAL)) AS Average_2016,
5   MAX(CAST(c14 AS REAL)) AS Max_2016,
6   MIN(CAST(c14 AS REAL)) AS Min_2016
7 FROM cleaned_defence_data;
```

The query is executed, and the result is displayed in a table with four columns:

Total_2016	Average_2016	Max_2016	Min_2016
208317.9	13887.859999999999	104158.95	-2836.88

On the right, the 'History' panel shows the execution history of the query, including the SQL code and the execution time (17:26:35).

10.

SQLite 0.14 beta (Mem...)

Table

- cleaned_defence_data
- demo

MariaDB

PostgreSQL

MS SQL

```
1 -- 10. Group and aggregate by Description
2 SELECT
3     c2 AS Description,
4     COUNT(*) AS Count_Entries,
5     SUM(CAST(c14 AS REAL)) AS Total_2016_Budget
6 FROM cleaned_defence_data
7 GROUP BY c2
8 ORDER BY Total_2016_Budget DESC;
```

Description	Count_Entries	Total_2016_Budget
Grand Total arate document	1	104158.95
Pay and Allowances of the Army	1	60548.18
Stores	1	16697.84
Works	1	8259
Rashtriya Rifles	1	5453
Pay and Allowances of Civilians	1	4878.8
Transportation	1	2828.1
Ex-Servicemen Contributory Health Scheme	1	2639
Other Expenditure	1	2298.18
Pay and Allowances and Miscellaneous Ex...	1	1172.2
National Pandit Centre	1	1012.90

SQLite

```
-- 10. Group and aggregate by Description
SELECT
    c2 AS Description,
    COUNT(*) AS Count_Entries
...
```

17:27:08

SQLite

```
-- 9. Aggregate: Sum, Avg, Max, Min of :
SELECT
    SUM(CAST(c14 AS REAL)) AS Total_2016_Budget
...
```

17:26:35

SQLite

```
-- 8. Count total number of rows
SELECT COUNT(*) AS Total_Rows
FROM cleaned_defence_data;
```

17:26:08

SQLite

```
-- 7. Sort results in descending order :
```

11.

SQLite 0.14 beta (Mem...)

Table

- cleaned_defence_data
- demo

MariaDB

PostgreSQL

MS SQL

```
1 -- 11. Rename columns in result using AS
2 SELECT
3     c2 AS Description,
4     CAST(c14 AS REAL) AS Budget_2016_2017
5 FROM cleaned_defence_data
6 ORDER BY Budget_2016_2017 DESC
7 LIMIT 5;
8
```

Description	Budget_2016_2017
Grand Total arate document	104158.95
Pay and Allowances of the Army	60548.18
Stores	16697.84
Works	8259
Rashtriya Rifles	5453

SQLite

```
-- 11. Rename columns in result using AS
SELECT
    c2 AS Description,
    CAST(c14 AS REAL) AS Budget_2016_2017
...
```

17:27:42

SQLite

```
-- 10. Group and aggregate by Description
SELECT
    c2 AS Description,
    COUNT(*) AS Count_Entries
...
```

17:27:08

SQLite

```
-- 9. Aggregate: Sum, Avg, Max, Min of :
SELECT
    SUM(CAST(c14 AS REAL)) AS Total_2016_Budget
...
```

17:26:35

12.

The screenshot shows a database management tool interface. On the left, there's a sidebar with a tree view showing a database named 'demo' with a table 'cleaned_defence_data'. The main area displays a SQL query in a dark-themed editor. The query is a SELECT statement with a CASE WHEN clause to categorize budget values. Below the query, the results are shown in a table with three columns: 'Scheme', 'Budget_2015_2016', and 'Budget_Category'. The results table has 10 rows of data. On the right, a 'History' panel shows a list of executed queries with their timestamps.

SQL Query:

```
1 -- 12. Conditional logic using CASE WHEN
2 SELECT
3   c3 AS Scheme,
4   CAST(c6 AS REAL) AS Budget_2015_2016,
5   CASE
6     WHEN CAST(c6 AS REAL) > 10000 THEN 'High Budget'
7     WHEN CAST(c6 AS REAL) BETWEEN 5000 AND 10000 THEN 'Medium Budget'
8     ELSE 'Low Budget'
9   END AS Budget_Category
10 FROM cleaned_defence_data;
```

Results Table:

Scheme	Budget_2015_2016	Budget_Category
Code	0	Low Budget
2076	49647.95	High Budget
2076	967.53	Low Budget
2076	4841.8	Low Budget
2076	2676.36	Low Budget
2076	494.96	Low Budget
2076	1781.39	Low Budget
2076	769.96	Low Budget
2076	13955.3	High Budget

13.

The screenshot shows the same database management tool interface. The SQL query in the editor is a SELECT statement with a subquery to find schemes with the highest budget compared to the average. The results table shows three rows of data. The 'History' panel on the right shows a list of executed queries with their timestamps.

SQL Query:

```
1 -- 13. Subquery to get schemes with highest 2016 budget compared to average
2 SELECT
3   c3 AS Scheme,
4   CAST(c14 AS REAL) AS Budget_2016_2017
5 FROM cleaned_defence_data
6 WHERE CAST(c14 AS REAL) > (
7   SELECT AVG(CAST(c14 AS REAL))
8   FROM cleaned_defence_data
9 )
10 ORDER BY Budget_2016_2017 DESC;
```

Results Table:

Scheme	Budget_2016_2017
	184158.95
2076	69548.18
2076	15697.84